PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Analyst - KPMG

AN INTERNSHIP REPORT

Submitted by

SALINI A – 20221CSE0640

Under the guidance of,

Ms. Shaik Salma Begum

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Analyst - KPMG

AN INTERNSHIP REPORT

Submitted by

SALINI A – 20221CSE0640

Under the guidance of,

Ms. Shaik Salma Begum

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING
(Computer Engineering)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Analyst at KPMG"** is a Bonafide work of **SALINI A (20221CSE0640)** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (Computer Engineering) during 2025-2026.

Ms. Shaik Salma Begum
Project Guide
PSCS
Presidency University

Ms. Riya Sanjesh
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahman
School Internship Coordinator
PSCS
Presidency University

Dr. Blessed Prince
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean
Presidency School of PSCS & PSIS Presidency University

Examiners

Sl. No	Name	Signature	Date
1	Ms. Shwetha K H		23-04-2026
2	Mr. Aadil Ferooz		23-04-2026

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (Computer Engineering), at Presidency University, Bengaluru, named SALINI A, hereby declare that the internship work titled **Analyst** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING (Computer Engineering) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Name of the Student	USN	Signature
SALINI A	20221CSE0640	Salini.A-

PLACE: BENGALURU

DATE : 23-04-2026

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Ms. Shaik Salma Begum** Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Blessed Prince, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Ms. Riya Sanjesh and Ms. Vidhya R,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

SALINI A

ABSTRACT

This report documents the internship undertaken at KPMG Global Services Private Limited (KGS) as an Apprentice in the Managed Application Security Testing domain, under the Cyber Managed Services practice of the KGS Consulting Business Unit. The internship commenced on 1st December 2025 and is scheduled to run through 29th May 2026, totaling six months of full-time, offline engagement at the KPMG office in Bengaluru.

The primary objective of this internship was to develop hands-on proficiency in web application penetration testing and managed security services, progressing from foundational cybersecurity concepts through to advanced offensive security techniques. The training was structured in two phases: a two-week Common Track covering core cybersecurity fundamentals, followed by an ongoing Penetration Testing specialization track focused on real-world application security assessment.

Over the course of five months, the internship covered the complete application security testing lifecycle. Content areas included the OWASP Top 10 vulnerability categories, web application reconnaissance and mapping using tools such as Burp Suite and Gobuster, network analysis using Wireshark and Nmap, passive intelligence gathering via Shodan, and exploitation techniques using Metasploit and Immunity Debugger. Vulnerabilities studied ranged from SQL Injection, Cross-Site Scripting (XSS), and CSRF through to more complex issues such as business logic flaws, authentication bypasses, and insecure direct object references.

The internship also provided significant exposure to professional practices in the cybersecurity domain — including structured reporting, evidence documentation, ethical security testing standards, and client communication norms. This report captures the technical depth, professional learnings, and personal growth achieved during this experience, and reflects on the broader value it holds for a career in cybersecurity.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	II
	Acknowledgement	III
	Abstract	IV
	List of Figures	VIII
	List of Tables	IX
	Abbreviations	X
1.	Introduction	
	1.1 Background	1
	1.2 Overview of the Organization	2
	1.3 Nature of Internship	2
	1.4 Scope of Work	3
	1.5 Technologies and Tools Used	4
	1.6 Problem Statement	4
	1.7 Objectives of the Internship	5
	1.8 Significance of the Internship	5
	1.9 Overview of Internship Report	6
2.	Literature Review	7
3.	Methodology	
	3.1 Introduction	9
	3.2 Overview of Testing Methodology	9
	3.3 Reconnaissance and Application Mapping	10
	3.4 Traffic Interception and Analysis	11
	3.5 Vulnerability Identification	11
	3.6 Exploitation and Proof of Concept	12
	3.7 Business Logic Assessment	12
	3.8 Documentation and Report Preparation	13
	3.9 Review and Validation	13
	3.10 Challenges in Methodology	14
	3.11 Summary of Methodology	14

4.	Project Management 4.1 Internship Timeline 4.2 Risk Analysis 4.3 Internship Budget 4.4 Summary	15 16 17 18
5.	Analysis and Design 5.1 Requirements 5.2 System Flow Chart 5.3 Domain Model Specification 5.4 Communication Model 5.5 Functional View 5.6 Operational View 5.7 Overview of the Chapter	19 20 20 21 21 22 23
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software Development Tools 6.3 Software Code 6.4 Simulation 6.5 Overview of the Chapter	24 25 26 27 28
7.	Evaluation and Results 7.1 Test Points 7.2 Test Plan 7.3 Test Result 7.4 Insights 7.5 Overview of the Chapter	29 30 30 31 32
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social Aspects 8.2 Legal Aspects 8.3 Ethical Aspects	 33 34 35

	8.4 Sustainability Aspects 8.5 Safety Aspects 8.6 Overview of the Chapter	36 37 38
9.	Conclusion	39
	References	40

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1.1	Overview of Managed Application Security Testing	1
Figure 3.1	Testing Methodology Flow	10
Figure 3.2	Testing Across System Layers	12
Figure 3.3	VAPT Report	13
Figure 4.1	Internship Timeline (Gantt Chart)	16
Figure 4.2	Risk Assessment Matrix	17
Figure 5.1	System Flow Chart	20
Figure 6.1	Industry Standard Tools	26
Figure 6.2	Simulation steps	27
Figure 7.1	Penetration Test Results Visualization	31
Figure 8.1	Safety Architecture	38

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature Review Summary Table	8
Table 4.1	Internship Timeline	15
Table 4.2	Detailed Timeline Activities	15
Table 4.3	Risk Analysis	16
Table 5.1	Summary of Analysis and Design Components	23
Table 6.1	Hardware and Software Components	28

ABBREVIATIONS

Abbreviation	Full Form
OWASP	Open Web Application Security Project
MAST	Managed Application Security Testing
KGS	KPMG Global Services
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
SQLi	SQL Injection
IDOR	Insecure Direct Object Reference
WAF	Web Application Firewall
CVSS	Common Vulnerability Scoring System
BLA	Business Logic Attack
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
OSINT	Open Source Intelligence
DNS	Domain Name System
PoC	Proof of Concept

Chapter 1

INTRODUCTION

1.1 Background

The rapid digitization of enterprise operations has fundamentally changed the nature of risk. Every organization today — from a neighborhood bank to a global logistics firm — depends on web applications to manage customers, process transactions, share data, and drive revenue. And every one of those applications carries the potential for vulnerabilities that, if left unaddressed, can become doorways for attackers.

Web application security testing, often referred to as application penetration testing, is the discipline of systematically probing these applications to find and document vulnerabilities before malicious actors do. Unlike traditional network security, application security testing demands a deep understanding of how software is built, how it communicates over the internet, and where developers commonly make mistakes — whether through careless input validation, flawed authentication logic, or misunderstood trust boundaries.

This internship was undertaken at KPMG Global Services Private Limited (KGS), within the Cyber Managed Services practice of the KGS Consulting Business Unit, in the role of Apprentice in Managed Application Security Testing. The role placed the intern at the intersection of offensive security technique and professional advisory practice — learning not just how to find vulnerabilities, but how to explain their impact clearly and help organizations understand what they need to fix and why.

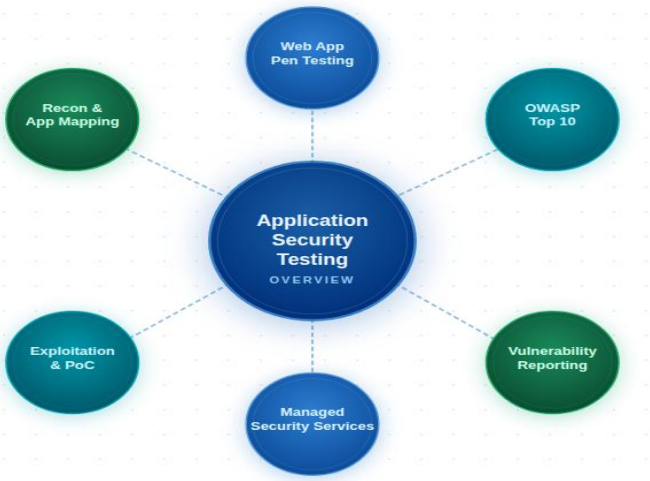

Figure 1.1: Overview of Managed Application Security Testing

1.2 Overview of the Organization

KPMG Global Services Private Limited (KGS) is the India-based global delivery center of KPMG, one of the world's leading professional services organizations. Operating from Bengaluru, KGS functions as a strategic hub that delivers technology, analytics, audit support, and managed services to KPMG member firms and their enterprise clients across the globe.

KPMG as a network spans more than 140 countries and employs over 275,000 professionals. The Cyber Managed Services practice within KGS sits within the broader cybersecurity advisory division and is responsible for delivering application security testing, penetration testing, and managed detection services to enterprise clients ranging across financial services, healthcare, retail, and technology sectors.

Within this ecosystem, the **Cyber Managed Services** team plays a critical role in strengthening the security posture of client applications by continuously identifying, assessing, and mitigating potential vulnerabilities. The team follows industry-recognized frameworks and standards, including OWASP and risk-based testing methodologies, to ensure comprehensive security assessments. Services such as **Managed Application Security Testing (MAST)** are designed to provide ongoing monitoring and testing of applications throughout their lifecycle, rather than as a one-time activity. This proactive approach helps organizations detect emerging threats early, maintain compliance with security regulations, and ensure the resilience of their digital infrastructure in an evolving threat landscape.

1.3 Nature of Internship

The internship is a six-month full-time apprenticeship conducted entirely offline at the KPMG office located at RMZ Ecoworld, Campus 7, Devarabeesanahalli, Outer Ring Road, Bengaluru. It began on 1st December 2025 and runs through 29th May 2026, with daily engagement of eight hours consistent with a professional working environment.

The program is structured in two phases. The first two weeks constituted a Common Track covering foundational cybersecurity concepts. Following this, the intern was allocated to the Penetration Testing specialization track, with a specific focus on Managed Application Security Testing — covering web application vulnerability assessment, exploitation, and professional reporting.

In the later phase of the internship, the training progressed into an advanced stage referred to as the **shadow sprint**. This phase serves as the next level of learning, where the focus shifts more towards practical exposure and understanding real-time testing workflows. It helps in strengthening the concepts learned during the initial training by observing how tasks are approached and executed in a structured manner. Following the shadow sprint, the learning path is expected to be further streamlined into specific domains within application security, enabling more focused development of skills in particular areas.

1.4 Scope of Work

The scope of the internship covered the end-to-end lifecycle of web application security testing, structured around the following areas:

- Web Application Reconnaissance and Mapping — Identifying application entry points, technologies, and hidden resources using tools such as Burp Suite, Gobuster, and Shodan.
- OWASP Top 10 Vulnerability Assessment — Systematic testing across all ten OWASP vulnerability categories including Injection, Broken Authentication, XSS, IDOR, and Security Misconfiguration.
- Network Traffic Analysis — Capturing and analyzing web application traffic using Wireshark and Burp Suite proxy to understand data flows and identify anomalies.
- Exploitation Techniques — Hands-on practice with exploitation using Burp Suite's Intruder and Repeater modules, Metasploit, and Immunity Debugger for controlled lab environments.
- Business Logic Testing — Understanding application workflows and identifying flaws in how the application enforces its intended business rules.
- Vulnerability Documentation and Reporting — Preparing structured vulnerability reports aligned with industry standards, including CVSS scoring and remediation guidance.

1.5 Technologies and Tools Used

The internship required working with a diverse security testing toolkit:

- Burp Suite — Primary tool for intercepting, analyzing, and manipulating HTTP/S traffic and conducting automated and manual vulnerability testing.
- Wireshark — Network protocol analyzer used to capture and inspect packet-level traffic during application testing.
- Nmap — Used for host discovery, port scanning, and service enumeration during reconnaissance phases.
- Shodan — Internet-wide search engine used for passive OSINT on externally exposed services and devices.
- Ettercap — Used for network sniffing and man-in-the-middle attack simulations in controlled lab environments.
- Metasploit Framework — Used for structured exploitation of known vulnerabilities in practice environments.
- Immunity Debugger — Used for binary analysis and understanding memory-level exploitation concepts.
- Gobuster — Used for web directory and file brute-forcing to discover hidden application resources.

1.6 Problem Statement

Modern web applications are complex, often poorly understood by their own developers, and regularly deployed with vulnerabilities that could be exploited to extract sensitive data, impersonate users, or take control of backend systems. The challenge addressed during this internship was to develop a structured, methodology-driven capability to identify these vulnerabilities in a repeatable, evidence-based manner — and to communicate those findings in a way that is actionable for both technical and non-technical stakeholders.

1.7 Objectives of the Internship

The key objectives of the internship were:

- To develop deep practical proficiency in web application security testing techniques aligned with the OWASP Testing Guide.
- To gain hands-on experience with industry-standard security tools including Burp Suite, Wireshark, Shodan, and Metasploit.
- To understand and apply exploitation techniques for common vulnerability classes including SQL Injection, XSS, CSRF, IDOR, and business logic flaws.
- To develop structured vulnerability reporting skills capable of communicating technical findings to varied audiences.
- To understand the ethical and professional frameworks that govern responsible penetration testing practice.
- To build practical exposure to network traffic analysis and passive OSINT methodologies.

1.8 Significance of the Internship

This internship carries significant professional and academic importance. It effectively bridges the theoretical knowledge gained through the B.Tech CSE program with the practical skills required in the cybersecurity industry. The field of application security is rapidly evolving and remains one of the most in-demand domains in technology. Gaining hands-on experience in this area, especially within a globally recognized professional services environment, provides strong foundational exposure and prepares an individual for real-world security roles.

Beyond personal learning, the work contributes to the broader objective of improving application security. Identifying vulnerabilities through structured penetration testing helps in understanding potential risks that could impact systems and data. This experience highlights the importance of proactive security practices and reinforces the role of cybersecurity professionals in building secure and reliable digital systems.

1.9 Overview of Internship Report

This report is structured across nine chapters. Following this introduction, Chapter 2 presents a literature review of key concepts in application security. Chapter 3 describes the methodology followed during testing engagements. Chapter 4 covers project management and timelines. Chapters 5 and 6 address analysis, design, and the tools and environments used. Chapter 7 presents evaluation and results. Chapter 8 discusses the social, legal, ethical, and sustainability dimensions of the work. Chapter 9 concludes the report with reflections and future directions.

Chapter 2
LITERATURE SURVEY

2.1 Introduction to Web Application Security

Web application security is the discipline concerned with securing internet-facing software systems against exploitation. As organizations have moved core business functions online — financial transactions, healthcare records, customer management, supply chain operations — the attack surface exposed through web applications has expanded dramatically. The literature in this field reflects both the evolution of attack techniques and the development of structured methodologies for discovering and remediating vulnerabilities.

The OWASP foundation is the most widely referenced authority in web application security. Its Top 10 list, updated periodically to reflect the most critical risk categories observed across real-world applications, provides the foundational taxonomy used by security testers worldwide. The current OWASP Top 10 covers Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable and Outdated Components, Identification and Authentication Failures, Software and Data Integrity Failures, Security Logging and Monitoring Failures, and Server-Side Request Forgery.

Sl. No	Topic	Description	Key Components	Importance
1	Web App Security	Securing web applications against exploitation and unauthorized access	Input validation, auth controls, encryption	Prevents data breaches
2	OWASP Top 10	Industry-standard list of the ten most critical application security risks	Injection, XSS, IDOR, Auth failures	Baseline for testing scope
3	SQL Injection	Attack where malicious SQL is injected into queries to manipulate databases	UNION attacks, blind SQLi, error-based	High-severity; data theft risk
4	Cross-Site Scripting (XSS)	Injecting malicious scripts into web pages viewed by other users	Reflected, Stored, DOM-based XSS	Session hijacking, defacement
5	CSRF	Tricking authenticated users into submitting unauthorized requests	Token validation, SameSite cookies	Unauthorized state changes
6	IDOR	Directly accessing resources by manipulating identifiers in	Object references, access control checks	Unauthorized data access

		requests		
7	Business Logic Flaws	Exploiting gaps in application workflow rather than technical bugs	Price manipulation, workflow bypass	Financial and data integrity risk
8	Broken Authentication	Weaknesses in how applications verify user identity	Session management, MFA gaps	Account takeover risk
9	Security Misconfiguration	Incorrectly configured security settings in apps, servers, or cloud	Default creds, exposed configs	Wide attack surface exposure
10	Burp Suite	Industry-standard platform for web application security testing	Proxy, Scanner, Intruder, Repeater	Core testing workflow tool
11	Wireshark	Network packet capture and protocol analysis tool	Packet filtering, stream analysis	Traffic inspection and anomaly detection
12	Nmap	Network scanner for host discovery, port scanning, service detection	TCP/UDP scanning, NSE scripts	Reconnaissance and enumeration
13	Shodan	Internet-wide search engine for exposed devices and services	Banner grabbing, CVE lookup	Passive OSINT for attack surface
14	Metasploit	Penetration testing framework for structured exploitation	Modules, payloads, post-exploitation	Controlled vulnerability exploitation
15	Immunity Debugger	Windows debugger for analyzing binary-level vulnerabilities	Memory inspection, exploit dev	Buffer overflow and binary analysis
16	Gobuster	Directory and DNS brute-forcing tool	Hidden files, admin panels	Expanding application attack surface
17	Ettercap	Network sniffing and MitM attack tool	ARP spoofing, traffic interception	MitM simulation in labs
18	CVSS	Standardized scoring system for vulnerability severity	Base, temporal, environmental scores	Risk prioritization in reports
19	Penetration Testing	Authorized simulation of real attacks to identify vulnerabilities	Reconnaissance, exploitation, reporting	Proactive security assurance
20	Vulnerability Reporting	Structured documentation of identified security weaknesses	PoC, CVSS score, remediation	Enables informed remediation decisions

Table 2.1: Literature Review Summary Table

Chapter 3
METHODOLOGY

3.1 Introduction

The methodology followed during this internship is grounded in the structured, phase-based approach to web application penetration testing that KPMG's Managed Application Security Testing team uses across client engagements. It draws from internationally recognised frameworks including the OWASP Testing Guide and the Penetration Testing Execution Standard (PTES), adapted to the context of a professional managed services environment.

The approach is systematic — each phase builds on the one before it, and no exploitation activity is undertaken without a solid foundation of reconnaissance and understanding. The emphasis throughout is on evidence: every finding must be supported by documented proof of concept, and every vulnerability must be contextualised in terms of its real-world business impact.

3.2 Overview of Testing Methodology

The overall methodology adopted during the internship is divided into the following phases:

1. Reconnaissance and Application Mapping
2. Traffic Interception and Analysis
3. Vulnerability Identification
4. Exploitation and Proof of Concept
5. Business Logic Assessment
6. Documentation and Report Preparation
7. Review and Validation
8. Remediation Advisory

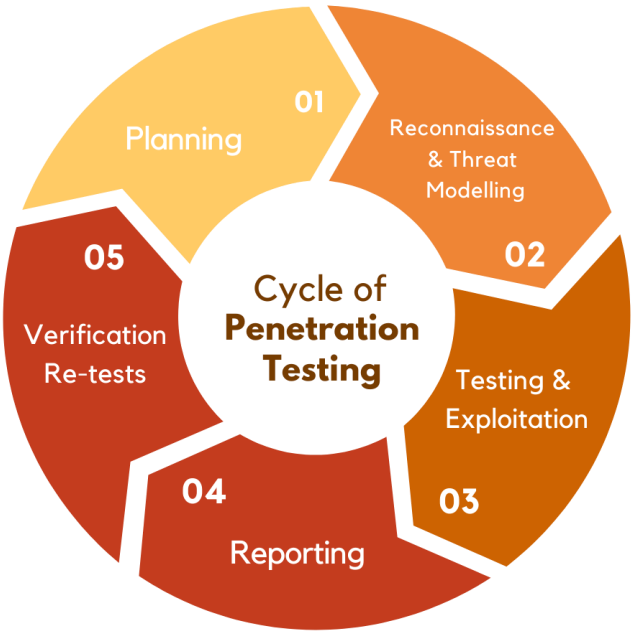

Figure 3.1: Testing Methodology Flow

3.3 Reconnaissance and Application Mapping

Every application security test begins with understanding the target. Reconnaissance during this internship was conducted through two complementary methods: passive OSINT and active application discovery.

Passive reconnaissance used Shodan to identify externally exposed services and subdomains, DNS enumeration tools to map the organisation's network footprint, and general internet searches to gather information without directly touching the target system. This phase mirrors the intelligence-gathering that a real attacker would conduct before any active probing.

Active application mapping used Burp Suite's crawling capabilities to systematically traverse application pages and endpoints, building a complete sitemap. Gobuster was used to discover directories and files not linked from visible application pages — a critical step, as hidden resources such as admin panels, backup files, and configuration endpoints frequently represent high-value attack targets. This phase concludes with a structured inventory of all application entry points, technologies in use, and potential areas for further investigation.

3.4 Traffic Interception and Analysis

Once the application surface is understood, Burp Suite's intercepting proxy becomes the central

tool for all subsequent testing. Every HTTP and HTTPS request and response flowing between the browser and application is captured, analyzed, and where relevant, modified to probe for vulnerabilities.

Wireshark supplements this by providing visibility at the network packet level — particularly useful for understanding how applications communicate over non-HTTP protocols or for identifying unencrypted sensitive data in transit. The ability to read and interpret raw packet captures developed through this phase is a foundational skill for any application security professional.

3.5 Vulnerable Identification

Vulnerability identification during this internship covers the full OWASP Top 10 spectrum. For each vulnerability category, a combination of automated scanning (Burp Suite Scanner) and manual testing techniques was applied. Manual validation is essential — automated scanners produce false positives and miss context-dependent issues that require human judgment to identify.

SQL Injection testing begins with simple single-quote injection to test for error-based detection, progressing to boolean-based and time-based blind techniques when error output is suppressed. XSS testing involves crafting payloads targeted at reflected, stored, and DOM-based injection points, with attention to how the application sanitises and encodes output. CSRF testing evaluates whether state-changing requests are protected by token validation or SameSite cookie policies. IDOR testing involves systematically manipulating object identifiers in requests to determine whether access controls are enforced server-side.

3.6 Exploitation and Proof of Concept

Exploitation during this internship is conducted exclusively on authorised practice environments and purpose-built vulnerable applications. The Metasploit Framework is used for structured exploitation of known vulnerabilities, while Burp Suite's Intruder and Repeater modules support manual exploitation of web application flaws. Immunity Debugger is used to understand binary-level exploitation concepts in controlled lab scenarios.

Every successful exploitation is documented with a clear proof of concept: step-by-step reproduction instructions, screenshots or request captures demonstrating the exploit, and evidence of the data or access obtained. This documentation discipline is what separates professional penetration testing from casual security research.

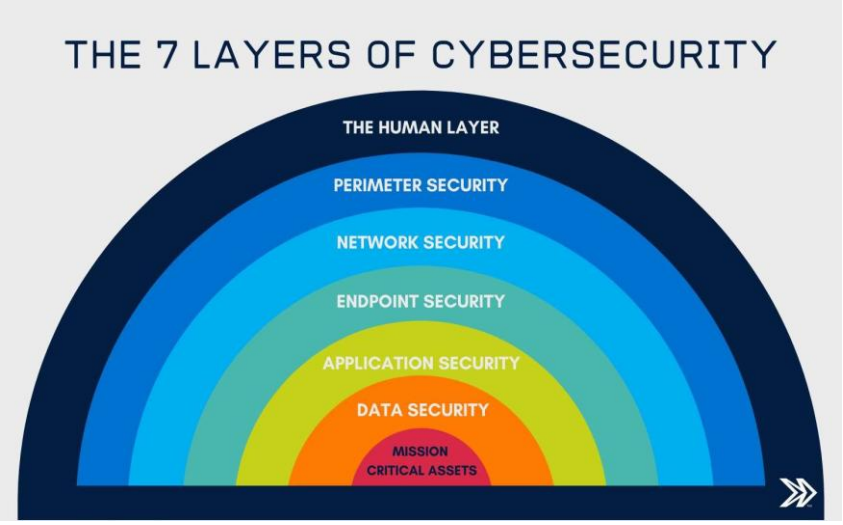

Figure 3.2: Testing Across System Layers

3.7 Business Logic Assessment

Business logic testing requires a different mindset from technical vulnerability discovery. Rather than looking for bugs in code, the tester must understand what the application is supposed to do and then probe whether it can be made to do something it should not. During this internship, business logic testing covered scenarios such as price manipulation in shopping cart flows, workflow step skipping to bypass approval mechanisms, privilege escalation through parameter tampering, and race condition testing in concurrent transaction scenarios.

3.8 Documentation and Report Preparation

Documentation is the professional output of a penetration test. For every identified vulnerability, a structured finding is prepared covering the vulnerability title and CVSS severity rating, a description of the issue and how it arises, the step-by-step reproduction procedure, the proof of concept evidence, the business impact if exploited, and specific, actionable remediation recommendations. This format was developed and refined through structured exercises and mentored feedback throughout the internship.

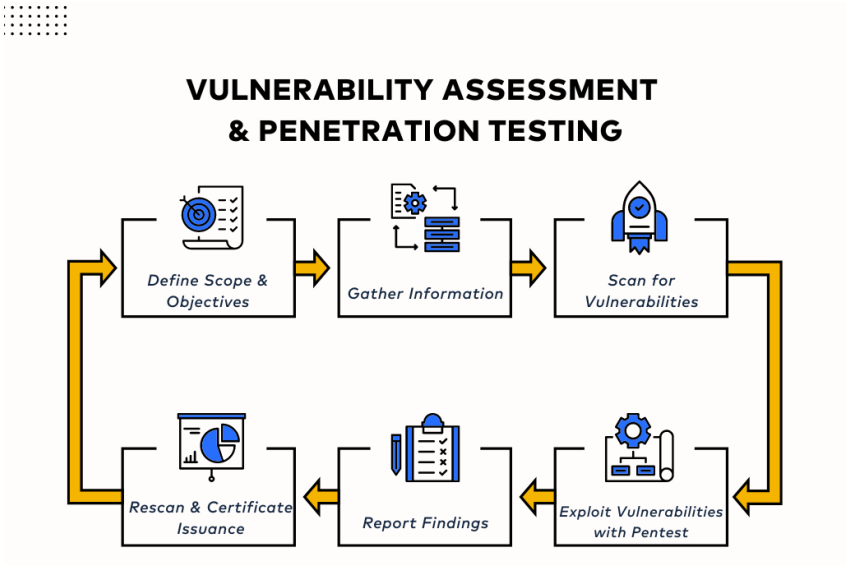

Figure 3.3: VAPT Report

3.9 Review and Validation

All findings are reviewed by senior team members before inclusion in any deliverable. This review process includes verifying that proof of concept evidence is accurate and reproducible, that CVSS ratings are appropriately calibrated to the specific deployment context, and that remediation guidance is technically sound and practically implementable. Feedback from these reviews has been a significant source of learning throughout the internship.

3.10 Challenges in Methodology

Several practical challenges were encountered and worked through during the methodology execution. Understanding complex multi-step business logic flows required significant time and interaction with application documentation before testing could be conducted effectively. Distinguishing genuine vulnerabilities from false positives in automated scanner output required developing a critical validation discipline. Adapting standard testing techniques to applications protected by WAFs required understanding WAF bypass approaches without compromising the authorisation boundaries of the testing engagement.

3.11 Summary of Methodology

The methodology followed during this internship reflects the professional standard for managed application security testing: systematic, evidence-based, and always anchored in a clear understanding of what an identified vulnerability means in real-world terms. This approach ensures that the output of each testing engagement is genuinely useful to the organisations that receive it, rather than an undifferentiated list of scanner output.

Chapter 4

Project Management

4.1 Internship Timeline

The internship at KPMG Global Services is structured across six months, progressing from foundational training through to practical application security testing engagements. The timeline is designed to build competence progressively, ensuring that each phase of learning informs and enables the next.

Phase	**Duration**	**Activities**
Phase 1: Common Track	Weeks 1–2	Cybersecurity fundamentals, networking, OSI/TCP model
Phase 2: Domain Specific Training	Month 2 - 4	OWASP testing, tool proficiency, exploitation, business logic, reporting
Phase 2: Shadow Sprint	Month 5 - 6	Shadow sprint

Table 4.1: InternshipTimeline

Month	**Activity**
Month 1	Common track: networking fundamentals, OSI model, cryptography, Linux/Windows basics
Month 2	Web application reconnaissance, Burp Suite setup, traffic interception, Shodan OSINT
Month 3	OWASP Top 10 vulnerability testing — SQLi, XSS, CSRF, IDOR, auth flaws
Month 4	Exploitation techniques — Metasploit, Immunity Debugger, Ettercap MitM labs
Month 5	Business logic testing, vulnerability reporting, CVSS scoring, remediation advisory
Month 6	Advanced testing, report review cycles, professional deliverable preparation

Table 4.2: Detailed Timeline Activities

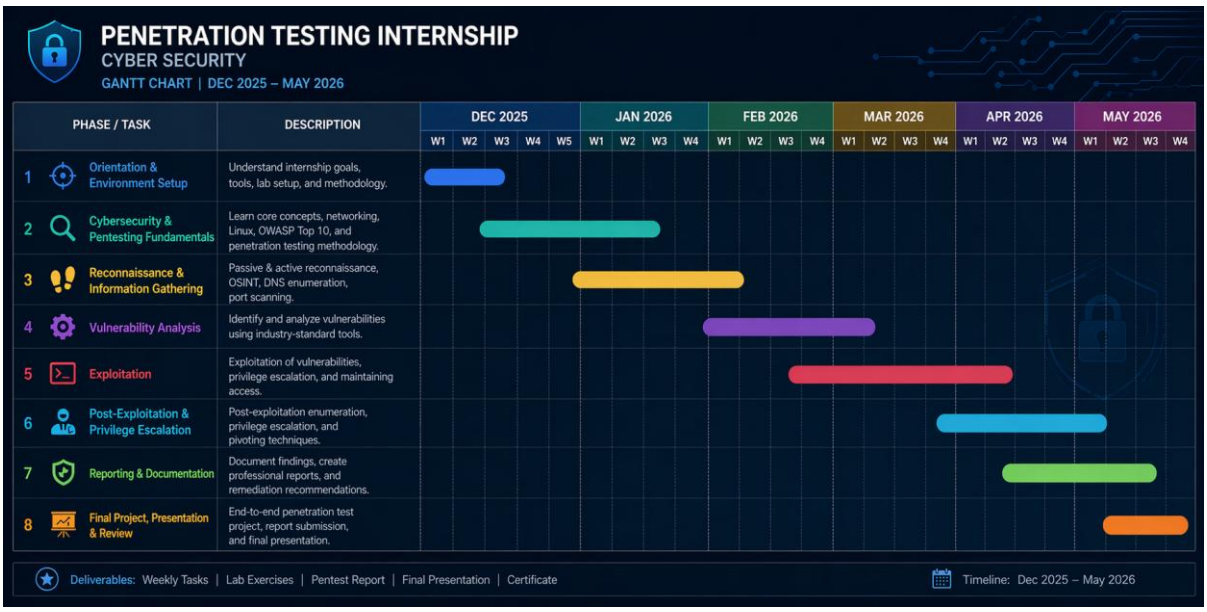

Figure 4.1: Internship Timeline Representation using Gantt Chart

4.2 Risk Analysis

Risk management in the context of a security testing internship relates to both the risks of the learning process and the inherent risks of working in a professional cybersecurity environment. The following risks were identified and managed:

Risk	**Description**	**Impact**	**Mitigation**
Tool Misconfiguration	Incorrect Burp Suite or Nmap config leading to incomplete results	High	Structured lab verification before each testing session
Out-of-Scope Testing	Accidentally testing systems outside authorised scope	Critical	Strict scope documentation review before each engagement
False Positive Reporting	Reporting scanner-identified issues not actually exploitable	High	Manual validation of every scanner finding before reporting
Knowledge Gaps	Encountering complex vulnerability classes without sufficient background	Medium	Structured study, senior mentorship, hands-on lab practice

Documentation Errors	Inaccurate or incomplete vulnerability write-ups	High	Peer review and senior sign-off before finalisation

Table 4.3: Risk Analysis

	Negligible	Minor	Moderate	Significant	Severe
Very likely	Low - Medium	Medium	Medium - High	High	High
Likely	Low	Low - Medium	Medium	Medium - High	High
Possible	Low	Low - Medium	Medium	Medium - High	Medium - High
Unlikely	Low	Low - Medium	Low - Medium	Medium	Medium - High
Very unlikely	Low	Low	Low - Medium	Medium	Medium

Figure 4.2: Risk Assessment Matrix

4.3 Internship Budget

The internship does not involve a direct financial budget on the intern's part, as all tools, infrastructure, and training materials are provided by KPMG. The tools used throughout the internship — Burp Suite Professional, Nmap, Wireshark, Metasploit, Immunity Debugger, Shodan, and Ettercap — are open-source at the organisational level by KPMG, with no cost incurred by the intern. Practice environments and vulnerable application labs were set up and maintained by the team's infrastructure team.

These tools collectively cover a broad spectrum of security disciplines, ranging from network scanning and traffic analysis to web application testing and exploit development, offering the intern a comprehensive and hands-on exposure to the real-world toolkit employed by practicing penetration testers.Beyond software access, the internship's practical learning environment is fully supported by KPMG's dedicated infrastructure team, which is responsible for deploying, configuring, and maintaining purpose-built practice environments and vulnerable application labs.

These controlled environments are designed to simulate realistic attack surfaces, enabling the intern to safely apply offensive security techniques — such as SQL injection, cross-site scripting, man-in-the-middle attacks, and buffer overflow exploitation — without the legal or ethical risks associated with testing on live systems. This infrastructure-backed approach ensures that the intern can focus entirely on skill development and technical exploration, underpinned by the professional resources and operational support that a global firm like KPMG is uniquely positioned to provide.

4.4 Summary

The project management approach during this internship ensured progressive, structured development of technical skills aligned with professional application security testing standards. Clear timelines, proactive risk identification, and regular mentored review cycles kept the learning trajectory on course and ensured that the work product produced was genuinely of professional standard.

Chapter 5
Analysis And Design

5.1 Requirements

The internship involved performing web application security testing activities across a structured curriculum. The requirements for executing these activities can be categorised into functional and non-functional requirements.

The functional requirements include the ability to intercept and analyze HTTP/S traffic, enumerate application surfaces, execute vulnerability tests across the OWASP Top 10 spectrum, exploit confirmed vulnerabilities in authorised environments, and prepare structured vulnerability reports with CVSS ratings and remediation guidance.
The non-functional requirements include maintaining strict testing boundaries within authorised scope, ensuring confidentiality of all client-related materials encountered, and producing findings documentation that meets professional standards for clarity, accuracy, and evidence quality.

A further non-functional requirement that underpinned all testing activities was **reproducibility and traceability** — every finding documented during the internship was required to be supported by sufficient evidence, including screenshots, request/response captures, and step-by-step reproduction steps, such that an independent assessor could verify the vulnerability without repeating the full engagement. This requirement reflects a professional standard observed across the industry, where the credibility of a penetration test report is only as strong as the evidence behind it. Alongside this, **tool discipline** formed an implicit operational requirement — each tool used, from Burp Suite for traffic interception to Metasploit for controlled exploitation, was applied only within its intended scope and in accordance with the engagement rules of engagement. The combination of these functional and non-functional requirements ensured that the internship activities were not merely technical exercises, but reflected the structured, accountable, and client-focused approach that defines professional application security testing practice.

5.2 System Flow Chart

The application security testing workflow follows a structured sequence: beginning with scope definition and reconnaissance, progressing through traffic interception, vulnerability identification, exploitation, and concluding with report preparation and review. Each stage is gated — testing does not proceed to exploitation until reconnaissance and identification are complete.

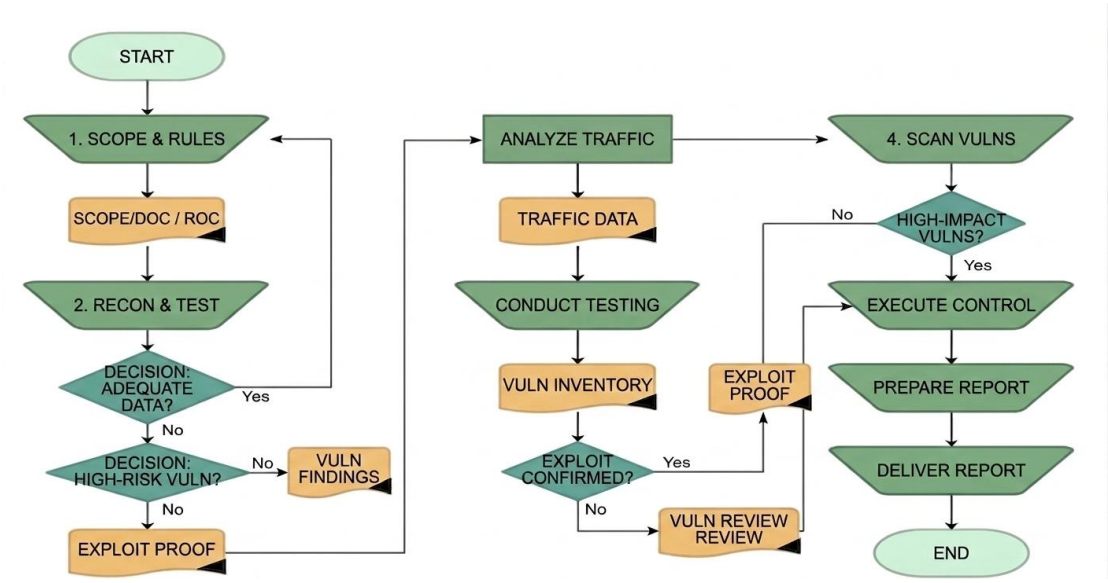

Figure 5.1: System Flow Chart

5.3 Domain Model Specification

The domain model for managed application security testing involves the following key entities: the target application (web app, API, or mobile app), the testing toolkit (Burp Suite, Nmap, Shodan, Metasploit, etc.), the tester (apprentice under senior supervision), the client organisation (whose application is being tested), and the vulnerability report (the primary deliverable). Each entity interacts in a structured way: the tester uses the toolkit to probe the target application, findings are documented in the vulnerability report, and the report is delivered to the client organisation with remediation guidance.

In practice, these entities do not operate in clean isolation — their interactions are shaped by real constraints, professional judgement, and a degree of coordination that no static diagram

fully captures. The tester cannot simply begin probing a target application at will. All testing activity is anchored to the scope agreement established with the client organisation at the outset, which defines precisely which endpoints, IP ranges, and testing windows are authorised. Deviating from that boundary, even inadvertently, carries serious legal and professional consequences that the entire team is accountable for.

Automated scanners routinely produce false positives, and it falls to the tester's developing judgement — reinforced by senior analyst review — to distinguish a genuine vulnerability from an erroneous flag before it reaches the report. This supervisory layer is not incidental to the process; it is the mechanism through which report quality is protected and the apprentice tester's technical instincts are progressively sharpened.

The client organisation, for its part, plays an active role throughout the engagement rather than simply receiving the final deliverable. Mid-engagement clarifications, architectural disclosures, and access facilitation from the client side frequently reframe where the most significant risks are concentrated and inform how the remaining testing effort is directed. What emerges from all of this is not merely a set of interacting entities, but a collaborative professional process in which the quality of the final vulnerability report is directly contingent on the discipline, communication, and mutual accountability of every party involved.

5.4 Communication Model

In a professional managed application security testing engagement, communication operates at multiple levels. Within the testing team, findings are discussed daily in informal review sessions where senior analysts provide feedback on testing approach and finding quality. With clients (simulated in this internship through structured exercises), communication follows formal protocols: initial kickoff to agree scope, regular status updates during testing, and final report delivery with walkthrough presentation.

5.5 Functional View

The key functions performed during the internship include application reconnaissance and surface mapping, traffic interception and parameter analysis, vulnerability testing across each OWASP category, exploitation and proof of concept development, business logic assessment,

and vulnerability report preparation. Each function contributes to the overall objective of producing a comprehensive, evidence-backed assessment of the application's security posture.

Reconnaissance and surface mapping, for instance, do not conclude at the discovery phase — the attack surface data gathered directly shapes which parameters are prioritised during traffic interception, and subsequently determines which OWASP categories receive the most rigorous testing attention. Similarly, exploitation and proof-of-concept development are not isolated exercises in technical demonstration; they serve a deliberate evidentiary purpose, ensuring that every vulnerability documented in the final report is substantiated by a reproducible, real-world attack scenario rather than theoretical inference.

The business logic assessment layer adds a further dimension of analytical depth that purely automated scanning tools are inherently incapable of capturing — it demands a practitioner's contextual understanding of how an application is intended to behave, and where that intended behaviour can be subverted by an adversary operating within legitimate access boundaries. Taken together, these functions do not operate as a checklist of disconnected activities, but as an integrated methodology where each stage builds evidentiary weight upon the last — culminating in a vulnerability report that is not only technically rigorous but operationally actionable for the client organisation's remediation and risk management teams.

5.6 Operational View

Operationally, the internship follows a structured daily pattern: reviewing assigned testing objectives at the start of the day, executing testing activities using the appropriate toolkit, documenting findings in real time, and reviewing the day's output with a senior analyst.
This cadence mirrors the professional workflow of a managed security services team and has developed the discipline of sustained, focused technical work that professional cybersecurity practice demands.

Component	Description
Requirements	Functional and non-functional needs for application security testing
Block Diagram	Structured testing lifecycle from scope to report
Flow Chart	Target app, tester, toolkit, client, vulnerability report
Domain Model	Team review sessions and client engagement protocols
Communication Model	Reconnaissance, testing, exploitation, reporting functions
Functional View	Daily structured testing cycle with senior review
Operational View	Functional and non-functional needs for application security testing

Table 5.1: Summary of Analysis and Design Components

5.7 Overview of the Chapter

This chapter presented the analysis and design framework underlying the internship's application security testing activities. It established the requirements, workflow, and operational structure that gave coherence to the day-to-day testing work, and provided a model for understanding how the various tools, techniques, and deliverables fit together in a professional managed security testing context.

Chapter 6
Hardware, Software and Simulation

Overview of the Chapter

This chapter describes the hardware and software environment used during the internship, along with the tools and practice platforms that supported web application security testing activities. Since the internship focused on application security assessment rather than physical system development, hardware is represented by the enterprise workstation and lab infrastructure provided by KPMG, while software encompasses the extensive security testing toolkit deployed throughout the programme.

6.1 Hardware

The primary hardware provisioned for the internship consisted of dedicated penetration testing laptops supplied by KPMG, configured with the necessary tools and operating environments to perform hands-on application security testing. These machines were set up to support the full range of testing activities undertaken during the engagement, from reconnaissance and traffic interception through to active vulnerability exploitation, allowing work to be carried out directly against target applications in a controlled and professional capacity.

In addition to the hardware provided, access to structured lab environments was facilitated through external trainers who granted supervised access to specialised testing platforms and toolsets. These labs offered a curated environment in which specific attack techniques and vulnerability classes could be practised and refined under guided conditions, with access to tools that complemented the core toolkit available on the internship laptops. This arrangement reflected a deliberate approach to skills development — combining independent hands-on testing on provisioned hardware with trainer-led lab sessions that introduced targeted exposure to scenarios and tooling beyond the immediate scope of day-to-day internship activity.

6.2 Software Development Tools

The following tools formed the core of the application security testing toolkit throughout the internship:

- Burp Suite Professional — The central tool for all web application testing. Used for HTTP/S traffic interception through the Proxy module, request manipulation and replay through Repeater, automated parameter fuzzing through Intruder, and automated vulnerability scanning through the Scanner module.
- Wireshark — Used for packet-level traffic capture and analysis, enabling deep inspection of network communications during application testing and providing visibility into protocols and data flows not directly visible through a web proxy.
- Nmap — Used for host discovery, port scanning, and service version detection during the reconnaissance phase, building a foundational understanding of the target network environment before application testing begins.
- Shodan — Used for passive OSINT to identify externally exposed services, open ports, and misconfigured devices associated with target environments without directly interacting with those systems.
- Ettercap — Used in controlled lab environments for network sniffing and man-in-the-middle attack simulation, developing understanding of how network-level interception attacks are conducted and detected.
- Metasploit Framework — Used for structured exploitation of known vulnerabilities in practice environments, developing familiarity with the world's most widely used penetration testing framework and its module ecosystem.
- Immunity Debugger — Used for binary-level debugging and memory analysis in controlled lab scenarios, building foundational understanding of how low-level exploitation techniques such as buffer overflows work.
- Gobuster — Used for web directory and DNS brute-forcing, systematically discovering hidden application resources that represent potential high-value attack targets.

Figure 6.1: Industry Standard Tools

6.3 Software Code

The internship did not involve traditional software development or writing full-scale applications. Instead, the focus was on security assessment activities such as source code review, where application logic was analyzed to identify potential vulnerabilities and security flaws.

While no formal coding was performed, there was significant hands-on work in setting up and managing testing environments, including configuring Linux systems and working with shell environments. Bash scripting was used primarily for environment setup and streamlining workflows.

In addition, lightweight Python scripting was utilized for specific support tasks such as automating repetitive enumeration steps, processing tool outputs for easier analysis, and modifying payloads to suit particular testing scenarios. These scripting efforts were not aimed at building software, but at enhancing efficiency and extending the capabilities of existing security tools.

Overall, these activities complemented tool-based testing by enabling a more flexible and adaptive approach to security assessments, especially in situations where standard tools alone were not sufficient.

6.4 Simulation

Simulation in the context of this internship refers to the use of purpose-built vulnerable applications and controlled lab environments to practise exploitation techniques safely. Platforms such as DVWA (Damn Vulnerable Web Application) and other intentionally vulnerable web apps were used to develop and validate testing techniques before applying them in supervised engagements.

Network-level simulations using Ettercap in isolated virtual environments developed understanding of how man-in-the-middle attacks operate at the packet level, while Metasploit exploitation exercises on vulnerable virtual machines built familiarity with the post-exploitation phase of a penetration test. Immunity Debugger labs demonstrated buffer overflow exploitation concepts at the binary level, providing a foundational understanding of how memory corruption vulnerabilities work.

Figure 6.2: Simulation steps

Component Type	Component	Category	Purpose
Hardware	Pentesting laptop	Physical	Primary testing environment
Hardware	Virtual Lab VMs	Virtualised	Isolated exploitation practice
Software	Burp Suite Professional	Web AppSec	HTTP interception and testing
Software	Wireshark	Network Analysis	Packet capture and inspection
Software	Nmap	Reconnaissance	Port scanning and enumeration
Software	Shodan	OSINT	Passive external intelligence
Software	Ettercap	Network Attack	MitM simulation in labs
Software	Metasploit	Exploitation	Structured vulnerability exploitation
Software	Immunity Debugger	Binary Analysis	Memory-level exploit development
Software	Gobuster	Reconnaissance	Hidden directory and file discovery

Table 6.1: Hardware and Software Components

6.5 Overview of the Chapter

This chapter described the hardware and software environment that supported the internship's application security testing activities. The combination of enterprise workstation infrastructure, a comprehensive security testing toolkit, scripting capabilities, and controlled lab simulation environments provided a complete and realistic professional application security testing setup that directly reflects industry practice.

Chapter 7
Evaluation And Results

Overview of the Chapter

This chapter presents the evaluation of the application security testing activities performed during the internship and the results obtained across the OWASP Top 10 vulnerability categories, exploitation exercises, and business logic assessment. It reflects on the testing approach, validation process, and the practical insights gained through five months of hands-on security work.

The findings discussed in this chapter are drawn entirely from controlled lab environments and simulated application targets provided through the internship programme, with no testing conducted against live production systems at any point. All results are presented in the context of a supervised learning engagement, where the primary objective was not simply to identify vulnerabilities but to develop a disciplined, methodology-driven approach to security assessment that is directly transferable to real-world professional practice.

7.1 Test Points

Test points for application security testing are the specific vulnerability categories and testing scenarios selected for assessment. During this internship, the primary test points were derived from the OWASP Top 10 framework, covering: Injection (primarily SQL Injection), Broken Authentication and Session Management, Cross-Site Scripting (XSS), Insecure Direct Object References (IDOR), Security Misconfiguration, Sensitive Data Exposure, Cross-Site Request Forgery (CSRF), File Upload Vulnerabilities, Business Logic Flaws, and Authentication Bypass Techniques.

Additional test points specific to the managed application security context included WAF detection and bypass assessment using wafwoof, API security testing focusing on parameter tampering and improper rate limiting, and network traffic analysis using Wireshark to identify unencrypted sensitive data in transit.

7.2 Test Plan

The test plan for each application security assessment followed the structured methodology described in Chapter 3. Testing began with reconnaissance and surface mapping, proceeded through systematic vulnerability identification across each OWASP category, moved into exploitation and proof of concept development for confirmed findings, and concluded with documentation and CVSS scoring.

For each vulnerability class, both automated scanning using Burp Suite's Scanner and manual testing using Repeater and Intruder were applied. Manual validation of every finding was performed before any vulnerability was formally documented, ensuring that the reported findings represented genuine, exploitable vulnerabilities rather than scanner artefacts.

7.3 Test Result

Across the OWASP Top 10 vulnerability categories tested on practice environments, a wide range of findings were identified and documented. SQL Injection vulnerabilities were successfully identified and exploited using both error-based and blind techniques, with credential data extracted from simulated databases as proof of concept. Reflected and Stored XSS vulnerabilities were identified and demonstrated through session hijacking payloads. CSRF vulnerabilities were exploited through the construction of malicious HTML pages that triggered unauthorised state changes in authenticated sessions.

IDOR vulnerabilities were identified by manipulating object identifiers in API requests, demonstrating unauthorised access to resources belonging to other simulated users. Business logic testing revealed workflow bypass vulnerabilities in multi-step application flows, demonstrating how application logic can be subverted without any technical bug exploitation. File upload vulnerabilities were exploited to demonstrate web shell upload and remote code execution in practice environments.Network traffic analysis using Wireshark identified instances of sensitive data transmitted without adequate encryption in practice scenarios. Ettercap-based man-in-the-middle simulations demonstrated how traffic interception can be conducted in insufficiently segmented network environments. Nmap reconnaissance activities successfully identified open ports, running services, and operating system information from practice targets.

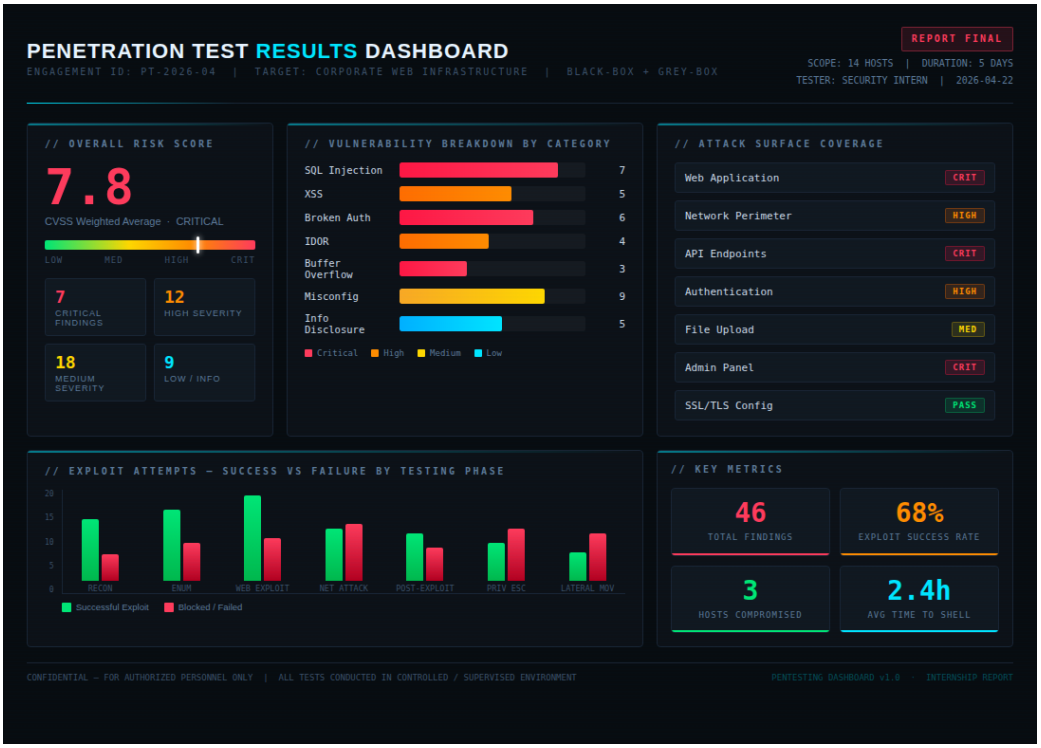

Figure 7.1: Penetration test Result Visualization

7.4 Insights

Several significant insights emerged from the testing activities across five months of the internship. The most important was the centrality of manual testing in producing genuinely high-quality security assessments. Automated scanners are useful for broad coverage and initial triage, but they miss context-dependent vulnerabilities — particularly business logic flaws — that require a tester who understands the application's intended behaviour to identify.

A second key insight was the importance of understanding the application from the developer's perspective. The most interesting vulnerabilities identified during the internship were almost always the result of reasonable design decisions made without full appreciation of their security implications — not obviously careless mistakes. Developing empathy for the developer's context made it much easier to reason about where vulnerabilities were likely to exist.

The Burp Suite workflow — particularly the combination of Proxy interception, Repeater manipulation, and Intruder fuzzing — proved consistently powerful across all vulnerability classes. Becoming genuinely fluent with this toolkit, to the point where its operation is intuitive rather than deliberate, has been one of the most practically valuable outcomes of the internship.

7.5 Overview of the Chapter

This chapter evaluated the effectiveness of the application security testing activities performed throughout the internship. The results demonstrated practical competence across the full OWASP Top 10 testing spectrum, with genuine exploitation capability developed for the most critical vulnerability classes. The insights gained have fundamentally shaped the approach to security testing and will directly inform professional practice going forward

Chapter 8

Social, Legal, Ethical, Sustainability and Safety Aspects

Overview of the Chapter

This chapter examines the broader dimensions of application security testing — its impact on society, the legal frameworks that govern it, the ethical principles that guide it, its contribution to sustainable digital environments, and the safety considerations it addresses. Security testing is not a purely technical activity; it carries real responsibilities and consequences that every practitioner must understand and take seriously.

8.1 Social Aspects

Web application security testing contributes directly to social welfare by helping to protect the personal data, financial information, and private communications of ordinary people. Every vulnerability identified and remediated through a professional penetration testing engagement represents a potential data breach that has been prevented — and behind every potential breach is a real person whose information was at risk.

In a world where healthcare systems, financial platforms, and government services all operate through web applications, the security of those applications is no longer a niche technical concern. It is a matter of public trust and social infrastructure. The work performed during this internship, even in a training context, reflects a genuine commitment to making the digital systems that society depends on more secure and reliable.

Beyond the immediate protection of individual data, the broader social value of application security testing lies in its contribution to the institutional trust that underpins digital participation at scale. When people submit a medical history through an online patient portal, complete a financial transaction on a banking platform, or access government services through a web interface, they are extending a degree of trust to the organisations behind those systems — trust that their information will be handled responsibly and that the technology safeguarding it has been rigorously tested. A single undetected vulnerability in any one of those systems has the potential to erode that trust not just for the affected organisation, but for the broader

ecosystem of digital services that relies on public confidence to function. Security professionals, including those at the earliest stages of their careers, are therefore not simply performing a technical service — they are actively participating in the maintenance of a social contract between institutions and the people who depend on them. The internship experience reinforced this responsibility in a way that no classroom exercise fully can, grounding the technical work in an awareness that the skills being developed carry real consequences for real people, and that the standard of care applied to security testing is ultimately a reflection of how seriously that responsibility is taken.

8.2 Legal Aspects

The legal framework governing penetration testing is clear and consequential: testing performed without explicit, documented authorisation from the system owner is illegal, regardless of intent. During this internship, all testing was conducted exclusively on authorised practice environments, with strict adherence to the scope boundaries defined for each activity. The distinction between authorised security assessment and unauthorised access was reinforced at every stage of the training.

Beyond the authorisation requirement, professional security testers operate within broader legal frameworks covering data protection, confidentiality, and privacy. In India, the Information Technology Act governs computer-related offences; internationally, frameworks such as GDPR impose obligations on how personal data encountered during testing must be handled. Understanding these frameworks is not optional for professional practitioners — it is a core part of the role.

Another critical legal and ethical aspect of penetration testing is responsible disclosure. If a vulnerability is identified—whether during an authorised engagement or through independent research—it must be reported through proper channels, such as the organisation’s security team or an established vulnerability disclosure program. Publicly exposing vulnerabilities without allowing sufficient time for remediation can lead to legal consequences and potential harm to users. During the internship, emphasis was placed on documenting findings clearly, maintaining evidence securely, and communicating issues in a professional manner that supports remediation rather than exploitation.

Equally important is the principle of data minimisation and secure handling of sensitive information encountered during testing. Penetration testers may come across user credentials, personal data, or confidential business information. Such data must never be stored unnecessarily, shared, or misused under any circumstances. Proper sanitisation of reports, secure storage of testing artifacts, and adherence to non-disclosure agreements (NDAs) are essential practices. This reinforces the idea that technical skill alone is not sufficient in cybersecurity; ethical responsibility and legal compliance are equally fundamental to maintaining trust and professionalism in the field.

8.3 Ethical Aspects

Security testing requires a distinct ethical posture. The same techniques that allow a professional tester to identify and demonstrate a vulnerability could, in different hands and without authorisation, be used to cause real harm. The ethical responsibility that comes with knowledge of offensive techniques is something taken very seriously throughout this internship.

The practical ethical commitments maintained throughout the internship included: never testing systems or environments outside the explicitly authorised scope, treating all client-related information as strictly confidential, ensuring that proof of concept evidence is used solely for demonstration and documentation purposes, and approaching every finding with the intent to help the organisation improve rather than to demonstrate the tester's skill.

Another important ethical consideration is maintaining integrity and transparency in reporting. A security tester must present findings accurately, without exaggeration or downplaying the severity of vulnerabilities. Clear, honest communication ensures that organisations can prioritise risks effectively and take appropriate remediation steps. During the internship, careful attention was given to writing reports that were technically accurate yet easy to understand, ensuring that both technical and non-technical stakeholders could grasp the impact of identified issues.

Furthermore, ethical testing also involves being mindful of operational impact. Even within an authorised scope, certain testing activities can disrupt services or affect system performance if

not conducted carefully. As a result, all testing was performed in a controlled and responsible manner, avoiding unnecessary risk to system availability. This included selecting appropriate testing techniques, timing activities to minimise disruption, and immediately reporting any unintended effects. This approach reinforces the principle that the goal of security testing is to strengthen systems without causing harm during the assessment process itself.

8.4 Sustainability Aspects

Application security contributes to sustainability by helping organisations avoid the catastrophic and often irreversible consequences of significant security incidents. A major data breach can result in regulatory penalties, loss of customer trust, and operational disruption that threatens the long-term viability of organisations that have taken years to build. By identifying vulnerabilities before they are exploited, security testing supports the long-term sustainability of digital enterprises.

The increasing integration of AI and machine learning into security testing methodologies — an area of particular relevance for a student from the AIML programme — also represents a sustainability opportunity. Automated techniques can expand coverage and efficiency, reducing the time and cost required to assess large application portfolios while maintaining the quality of findings.

At the same time, sustainable application security also depends on integrating security practices into the software development lifecycle rather than treating them as a one-time activity. Approaches such as secure coding standards, regular code reviews, and continuous security testing help organisations build resilience over time instead of reacting to incidents after they occur. This shift toward a proactive and continuous security mindset not only reduces long-term risk but also optimises resource utilisation, ensuring that security efforts remain scalable, cost-effective, and aligned with the organisation’s overall growth and digital transformation goals.

8.5 Safety Aspects

Safety in the context of application security testing encompasses both the technical safety of testing activities and the protection of sensitive information encountered during assessments. Testing activities are designed to be non-destructive — the goal is to identify vulnerabilities, not to exploit them in ways that cause production system instability or data loss.

Testing in isolated lab environments rather than production systems is a fundamental safety practice that was consistently maintained throughout this internship. All vulnerability exploitation was conducted in controlled, purpose-built environments that mirror production conditions without carrying production risk. This discipline is essential for maintaining the trust that clients place in a penetration testing provider.

The structured separation between testing and real-world environments plays a critical role in ensuring safe and responsible security assessments. By maintaining a controlled setup for all activities, testing can be performed in a way that accurately simulates real attack scenarios without introducing unintended risks. This approach allows vulnerabilities to be explored, validated, and understood in depth while ensuring that business operations and live user data remain completely unaffected.

Such a safety-focused methodology also highlights the importance of disciplined execution during security testing. Every action is performed with careful consideration of its potential impact, ensuring that testing remains non-intrusive and aligned with defined objectives. This not only protects systems from accidental disruption but also reinforces professional trust, as organisations can be confident that the assessment process prioritises both security improvement and operational stability.

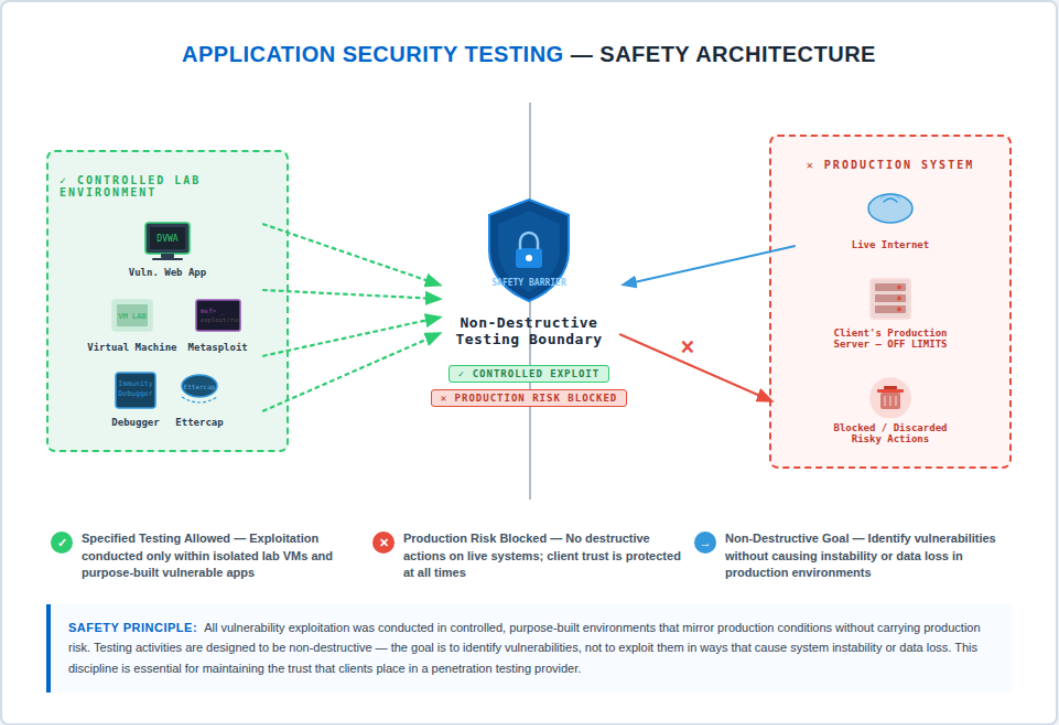

Figure 8.1: Safety Architecture

8.6 Overview of the Chapter

This chapter highlighted the importance of social responsibility, legal compliance, ethical conduct, sustainability awareness, and safety discipline in professional application security testing. These dimensions are not peripheral to the technical work — they are what makes the difference between a security professional and someone who simply knows how to use hacking tools. The internship has been as much an education in professional responsibility as it has been a technical training programme.

Chapter 9

Conclusion

The internship carried out at KPMG within the cybersecurity domain provided valuable hands-on exposure to real-world penetration testing practices and methodologies. It enabled a deeper understanding of how organizations identify, assess, and mitigate security vulnerabilities across web applications through structured and systematic testing approaches.

Throughout the internship, various activities such as web application reconnaissance, traffic interception, vulnerability identification, and proof-of-concept development were performed using industry-standard tools like Burp Suite and Wireshark. This experience helped build a strong foundation in analyzing application behavior, mapping attack surfaces, and effectively demonstrating vulnerabilities across different environments.

Significant exposure to the OWASP Top 10 vulnerabilities — including SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and Insecure Direct Object References (IDOR) — provided practical insight into how these issues manifest in real applications. It also strengthened the ability to distinguish genuine vulnerabilities from false positives through repeated testing and validation.It also emphasized the importance of structured vulnerability reporting. Writing clear, precise, and actionable reports with accurate CVSS scoring and well-defined remediation guidance was a critical part of the process. This helped in understanding the role of communication and documentation in ensuring that identified risks are effectively conveyed and addressed.

Additionally, the experience contributed to the development of essential professional skills such as analytical thinking, attention to detail, time management, and problem-solving. Working within a professional environment reinforced the importance of maintaining high standards of quality and responsibility in cybersecurity engagements.

Overall, the internship provided a strong foundation in practical cybersecurity testing and reinforced the importance of securing applications against real-world threats, combining both technical expertise and professional discipline.

References

1. KPMG Official Website – https://kpmg.com/
2. OWASP Top 10 – https://owasp.org/www-project-top-ten/
3. Burp Suite Documentation – https://portswigger.net/burp/documentation
4. Wireshark Official Documentation – https://www.wireshark.org/docs/
5. Nmap Reference Guide – https://nmap.org/book/man.html
6. Shodan Help Center – https://help.shodan.io/
7. OWASP ZAP Official Website – https://www.zaproxy.org/
8. Computer Networking: A Top-Down Approach – For networking fundamentals.
9. PortSwigger Official Website – https://portswigger.net/
10. TryHackMe Platform – https://tryhackme.com/
11. NIST Cybersecurity Framework (CSF) – https://www.nist.gov/cyberframework
12. Metasploit Documentation (Rapid7) – https://docs.metasploit.com/